Diplomatura en programación web full stack con React JS

Módulo 5:

Base de datos, integración con Node.js

Unidad 4:

Integración de Node.js con base de datos (parte 2)



Presentación

A partir de esta unidad comenzamos con la creación del panel de administración del sitio que armamos previamente. Este proyecto nos permitirá vincular todos los conceptos que fuimos incorporando en las unidades anteriores. Para este fin vamos a comenzar programando las funcionalidades de login y logout, así como también incorporaremos el concepto de páginas o rutas privadas.



Objetivos

Que los participantes logren…

- Crear la base de datos y las tablas necesarias para manejar usuarios.
- Comprender los procesos de autenticación de usuarios y la destrucción de sesiones.
- Implementar mecanismos para autorizar o no el acceso a ciertas rutas de la aplicación.



Bloques temáticos

1. Login.
2. Logout.
3. Rutas privadas.

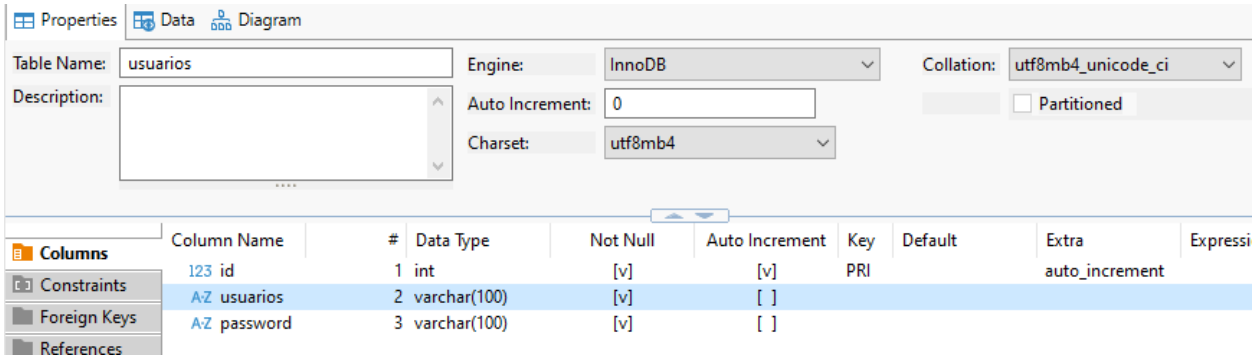
1. Login

Creamos la base de datos por ej: **transportes**. Para el charset o juego de caracteres seleccionamos **utf8mb4_unicode_ci** que nos permitirá guardar acentos y caracteres especiales sin dificultad.

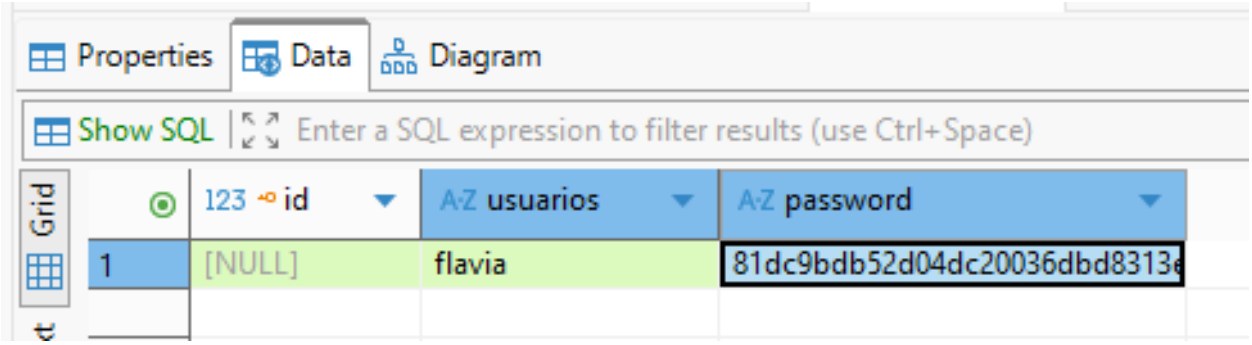
Con nuestra base de datos ya definida vamos a crear nuestra primera tabla, la cual va a almacenar a los usuarios que vamos a permitir ingresar a nuestro panel de administración.

La tabla se llamará **usuarios** y contará con 3 campos:

- **id**: un entero que será nuestra clave primaria, Not Null, Auto Increment
- **usuario**: un varchar de máximo 100 caracteres, Not Null.
- **password**: otro varchar de máximo 100 caracteres, Not Null

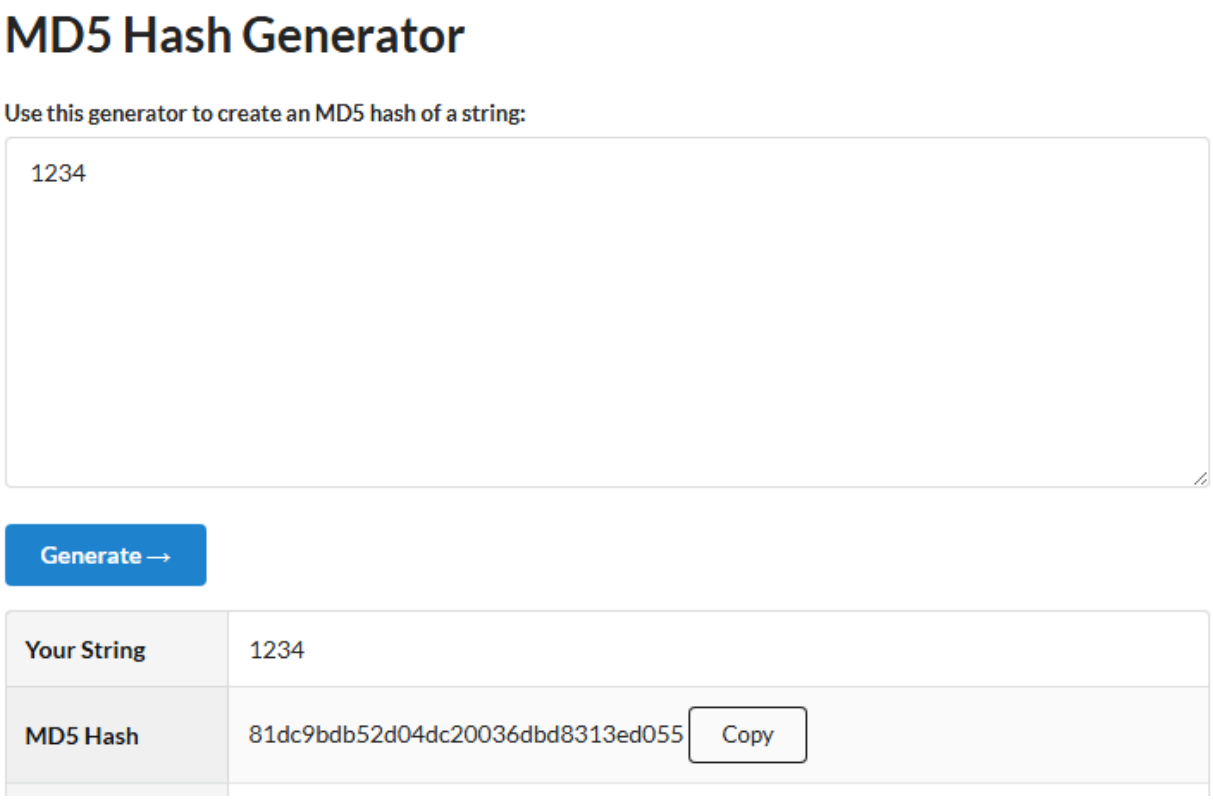


Cuando aceptemos estos valores tendremos acceso a la opción de **DATA**, que nos permitirá agregar registros a nuestra tabla.



Es importante que en el campo password este **encriptada (MD5)**. Esta función es un algoritmo de encriptación de una sola vía, lo que significa que una vez encriptado el valor no puede ser revertido a un valor legible. Esto lo hacemos por cuestiones de seguridad, ya que si alguna persona llegara a tener acceso a nuestra base de datos, si no encriptan las contraseñas, tendría acceso a las claves de todos los usuarios de nuestra aplicación.

Puede generar el password desde: https://www.md5hashgenerator.com/



Una vez finalizada la creación de nuestra base de datos instalaremos las dependencias necesarias en nuestro proyecto para poder comenzar con la programación del login.

Instalaremos **express-session, mysql2, útil y md5**. A excepción de md5, el resto ya las hemos utilizado en ejemplos previos y conocemos su funcionamiento. md5 cumple la función de encriptar cadenas de texto usando el mismo algoritmo que usamos para encriptar la contraseña de nuestro usuario. En este caso la usaremos para encriptar lo que el usuario ingrese en el formulario de login y de esa manera comparar las 2 cadenas encriptadas para asegurarnos que el usuario sea quien dice ser.

Completados todos los pasos de preparación estamos listos para comenzar con la programación del login de nuestro panel de administración.

Paso 1

Crearemos el archivo routes/admin/login.js donde iremos incorporando las rutas relacionadas al proceso. Por el momento solo incluiremos este código.

```
var express = require('express');
var router = express.Router();


router.get('/', function (req, res, next) {
  res.render('admin/login', {
    layout: 'admin/layout',
  });
});

module.exports = router;
```

Paso 2

Importamos nuestro manejador de rutas en **app.js** y lo configuramos para que se encargue de las rutas que comienzan con **/admin/login**

```
var loginRouter = require('./routes/admin/login');


app.use('/admin/login', loginRouter);
```

Paso 3

Creamos el archivo **views/admin/layout.hbs** que será el nuevo layout base para nuestro administrador.

```
<!DOCTYPE html>
<html>

<head>
    <title>{{title}}</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
rel="stylesheet"
        integrity="sha384-sRIl4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB"
crossorigin="anonymous">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/7.0.1/css/all.min.css"
        integrity="sha512-
2SwdPD6INVrV/lHTZbO2nodKhrnDdJK9/kg2XD1r9uGqPo1cUbujc+IYdlYdEErWNu69gVcYgdxlmVmzTWnetw=="
        crossorigin="anonymous" referrerpolicy="no-referrer" />
    <link rel='stylesheet' href='/stylesheets/style.css' />
</head>

<body>
    <header style="text-align:center;">
        <div class="holder">
            <img src="/images/logo.png" width="100" alt="Transportes X">
            <h1>Transportes X</h1>
        </div>
    </header>

    {{{body}}}

    <footer>
        <p>Diseñado por Flavia Ursino - &copy; año</p>
    </footer>


    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
        integrity="sha384-FKyoEForCGlyvwx9Hj09JcYn3nv7wiPVlz7YYwJrWVcXK/BmnVDxM+D2scQbITxI"
        crossorigin="anonymous"></script>
</body>

</html>
```

Paso 4

Creamos el archivo **views/admin/login.hbs** que albergará el formulario de login para acceder al panel.

```
<div class="container" style="margin-top:100px;">
    <div class="row">
        <form action="/admin/login" method="post">
            <div class="mb-3 row">
                <div class="col-sm-10">
                    <input type="text" class="form-control" id="inputUser"
                           placeholder="Usuario" name="usuario">
                </div>
            </div>
            <div class="mb-3 row">
                <div class="col-sm-10">
                    <input type="password" class="form-control" id="inputPassword"
                           placeholder="Contraseña" name="password">
                </div>
            </div>
            <button type="submit" class="btn btn-primary">Ingresar</button>
        </form>
    </div>
</div>
```

Paso 5

Para este paso es importante que tengamos configurados los archivos **.env** y **bd.js** con la configuración correcta para conectarnos a nuestro servidor de MySQL local.

Debido a que vamos a empezar a trabajar con más archivos y a realizar más operaciones involucrando a la base de datos, y dichas operaciones se utilizarán en varios archivos, vamos a emplear el concepto de modelos de la arquitectura MVC.

Los modelos serán archivos donde crearemos funciones que nos permitan consultar, crear, actualizar o eliminar un tipo de dato en particular. De esta forma, al importar o requerir estas funciones en otros archivos como nuestros controladores tendremos acceso a la información sin necesidad de tipear varias veces lo mismo.

El primer archivo que crearemos con esta finalidad es **models/usuariosModel.js**, cuyo contenido será el siguiente:

```
var pool = require('./bd');
var md5 = require('md5');


async function getUserByUsernameAndPassword(user, password) {
    try {
        var query = "select * from usuarios where usuario = ? and password = ? limit
1";
        var rows = await pool.query(query, [user, md5(password)]);
        return rows[0];
    } catch (error) {
        throw error;
    }
}


module.exports = { getUserByUsernameAndPassword }
```

En este archivo vemos empleado el uso de la estructura **try/catch** que sirve para interceptar excepciones y que estas no lleguen al usuario. Las excepciones son errores que nuestra aplicación arroja y que podemos ir interceptando con distintos bloques try/catch y poder actuar en consecuencia.

La función **getUserByUsernameAndPassword** recibe como parámetros el nombre de usuario y la contraseña y devuelve, en caso de encontrar una coincidencia en la tabla usuarios de nuestra base de datos, la fila correspondiente como un objeto.

Paso 6

En el archivo **routes/admin/login.js** agregaremos el controlador necesario para capturar los datos enviados por el formulario. También vamos a importar el archivo usuariosModel para poder utilizar la función que creamos.

```
var usuariosModel = require('./../../models/usuariosModel');


router.post('/', async (req, res, next) => {
  try {
    var usuario = req.body.usuario;
    var password = req.body.password;

    var data = await
usuariosModel.getUserByUsernameAndPassword(usuario, password);

    if (data != undefined) {
      req.session.id_usuario = data.id;
      req.session.nombre = data.usuario;
      res.redirect('/admin/novedades');
    } else {
      res.render('admin/login', {
        layout: 'admin/layout',
        error: true
      });
    }

  } catch (error) {
    console.log(error);

  }
})
```

Como podemos ver, estamos capturando los valores enviados por el formulario y los almacenamos en variables que después pasamos como argumentos del método que creamos en el modelo de usuarios. En caso de que la respuesta sea diferente a undefined (la combinación de usuario y contraseña existe) vamos a crear las variables de sesión id_usuario y nombre y asignaremos su valor a los

devueltos por la consulta a la base de datos. Por último redirigimos al usuario a hacia la ruta **/admin/novedades**.

Si no llega a haber coincidencia, ya sea porque el usuario escribio mal algún dato o porque directamente no existe esa combinación, haremos un render del template de **admin/login.hbs** especificando el nuevo layout de admin/layout.hbs y le enviaremos la variable error seteada en true para poder notificar al usuario de que hubo un error en el proceso de autenticación, para lo cual incluiremos el siguiente código en el template:

```
{{#if error}}
     <p>Usuario o clave incorrecto.</p>
{{/if}}
```

Paso 7

Creamos el archivo **routes/admin/novedades.js** con el siguiente contenido para mostrar el template correspondiente:

```
var express = require('express');
var router = express.Router();

router.get('/', function (req, res, next) {
   res.render('admin/novedades', {
    layout: 'admin/layout',
    usuario: req.session.nombre,
    });
});

module.exports = router;
```

En este archivo pasamos a la vista el nombre de usuario registrado en la sesión que acabamos de iniciar.

Paso 8

Creamos la vista para novedades en **views/admin/novedades.hbs** e ingresamos este contenido.

```
<div class="container" style="margin:50px auto">
    <div class="row">
        <div class="col">
            <p class="text-right">Hola {{ usuario }} ! </p>
        </div>
    </div>
</div>
```

2. Logout

Paso 1

En el archivo **routes/admin/login.js** creamos la función que se encargará de terminar la sesión del usuario.

```
router.get('/logout', function (req, res, next) {
  req.session.destroy();
  res.render('admin/login', {
    layout: 'admin/layout'
  });
});
```

El método **destroy()** del objeto sesión es el encargado de eliminar por completo la sesión del usuario y todos los valores almacenados en ella.

Paso 2

Modificamos nuestro archivo **views/admin/novedades.hbs** e incluimos un link a nuestra nueva función de logout.

```
<div class="row">
    <p class="text-end">Hola {{usuario}} ! <a class="btn btn-danger"
href="/admin/login/logout">Cerrar sesión <i class="fas fa-sign-out-alt"></i></a></p>
</div>
```

3. Rutas privadas

El proceso de autorización es el cual en el que se verifica no solo que el usuario haya iniciado correctamente (proceso de autenticación) sino que además tendremos la posibilidad de verificar (en casos más avanzados) que el usuario logueado tenga los permisos necesarios para realizar la acción que esté solicitando.

En nuestro ejercicio simplemente verificaremos que el usuario haya pasado por el proceso de autenticación, el cual deja como resultado algunas variables de sesión que podemos, cuya existencia podemos chequear en nuestras rutas para determinar si está o no logueado.

Paso 1

En el archivo **app.js** vamos a crear una nueva función y vamos a asignarla a una variable llamada secured.

```
secured = async(req,res,next) => {
    try{
        console.log(req.session.id_usuario);
        if(req.session.id_usuario){
            next();
        } else {
            res.redirect('/admin/login');
        }

    }catch(error){
        console.log(error);

    }
}
```

Esta función, que utilizaremos como **middleware** en todas las rutas que deseemos proteger, será la encargada de verificar que exista la variable de sesion id_usuario. En caso de encontrarla ejecutará la función next() que pasa el control de la petición a la siguiente función.

Si un usuario ingresa a una ruta protegida sin haber iniciado sesión correctamente interrumpimos el proceso de la petición y lo redirigimos hacia la pantalla de login.

Para implementar esta nueva función en nuestras rutas simplemente debemos ponerla antes del manejador de ruta al momento de declararla.

```
app.use('/admin/novedades',secured, adminNovedadesRouter);
```

Conclusión

En resumen, hemos aplicado los conceptos aprendidos, logramos armar un login, un logout e implementamos rutas privadas utilizando middleware. Enfatizamos la importancia de practicar distintas situaciones y que los alumnos utilicen sus habilidades para crear y experimentar en sus propios proyectos.

En la próxima unidad veremos…

Comenzaremos a integrar todos los conceptos aprendidos hasta ahora para empezar con la creación de un módulo de administración para nuestro proyecto.

Bibliografía utilizada y sugerida

Artículos de revista en formato electrónico:

Google Cloud. (s.f.). Especificar dependencias en Node.js. https://cloud.google.com/functions/docs/writing/specifying-dependencies-nodejs?hl=es-419

npm. (s.f.). npmjs.com: Node Package Manager. https://www.npmjs.com/